Split Payment Platform

Arquitetura de microserviços orientada a eventos para **simulação e processamento de Split Payment** conforme a **LC 214/2025** e **LC 227/2026** (reforma tributária do consumo: IBS e CBS).

> **Aviso.** As referências a artigos de lei (Art. 31, 32, 33 e 471-D) vêm do blueprint original e devem ser validadas por um tributarista antes de uso fora de simulação. Este projeto é uma **simulação técnica**, não um sistema de recolhimento oficial.

Sumário

1. Visão geral
2. Arquitetura
3. Componentes e responsabilidades
4. Fluxo de processamento
5. Modelagem de dados
6. Contratos (API e eventos)
7. Regra de cálculo do Split
8. Infraestrutura (Docker Compose)
9. Resiliência, idempotência e consistência
10. Implementação de referência (.NET 8)
11. Performance e baixo nível
12. Pontos de atenção e melhorias
13. Roadmap

---

1. Visão geral

Quando uma nota fiscal é emitida, o valor da operação é dividido (*split*) em duas partes:

- **Valor líquido** para o fornecedor (merchant);
- **Tributos** (IBS e CBS), retidos e direcionados ao ente arrecadador (Comitê Gestor do IBS e CBS, o CGIBS, e à CBS federal).

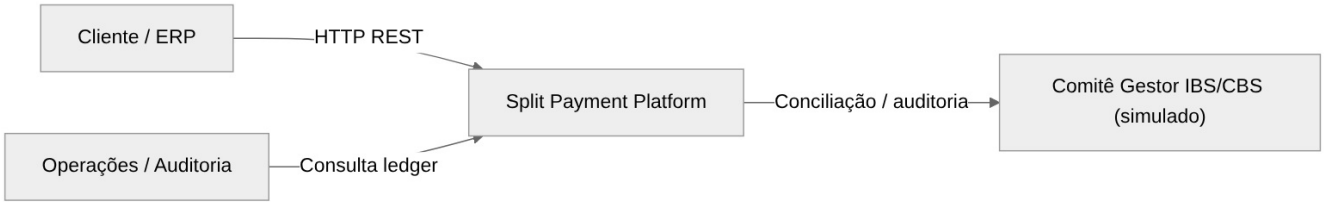
O sistema desacopla a emissão do documento da retenção e da conciliação por meio de um barramento de eventos (Kafka).

Objetivo	Como é atendido
Desacoplar emissão e retenção	Evento `InvoiceCreated` no Kafka
Precisão monetária	`decimal` (C#) e `DECIMAL(18,4)` (SQL)
Sem dupla retenção	Idempotência por `InvoiceId`
Auditoria	Ledger relacional no CGIBS Gateway
Histórico flexível de payloads	MongoDB

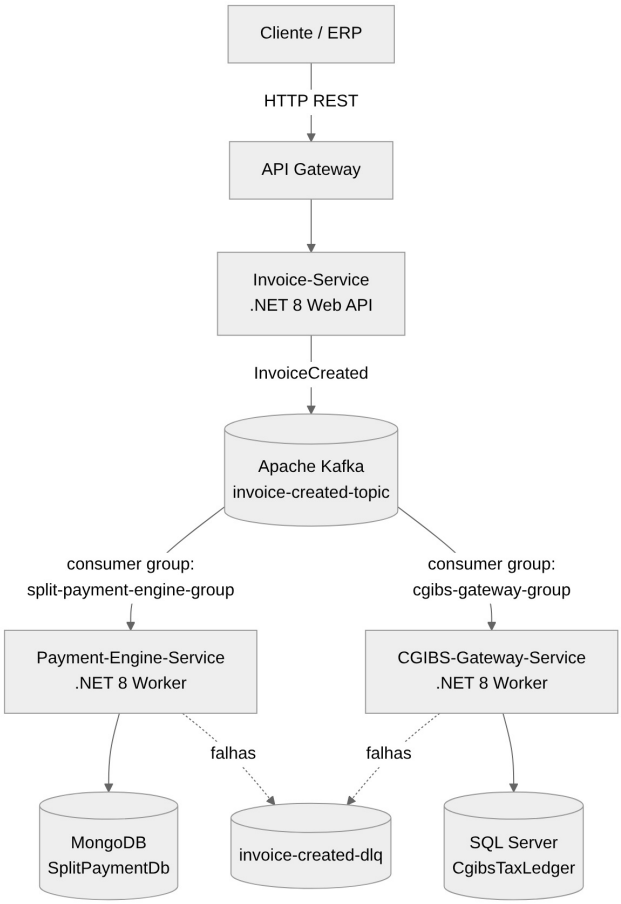
---

2. Arquitetura

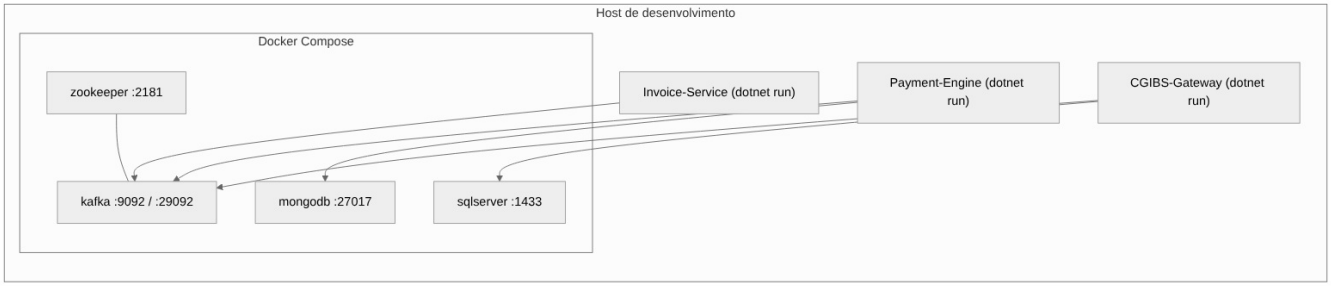
2.1 Visão de contexto (C4 nível 1)



2.2 Visão de containers (C4 nível 2)



2.3 Topologia de implantação (desenvolvimento)



Os serviços .NET rodando no host usam `localhost:9092`. Se forem containerizados, usam `kafka:29092` (ver seção 8).

---

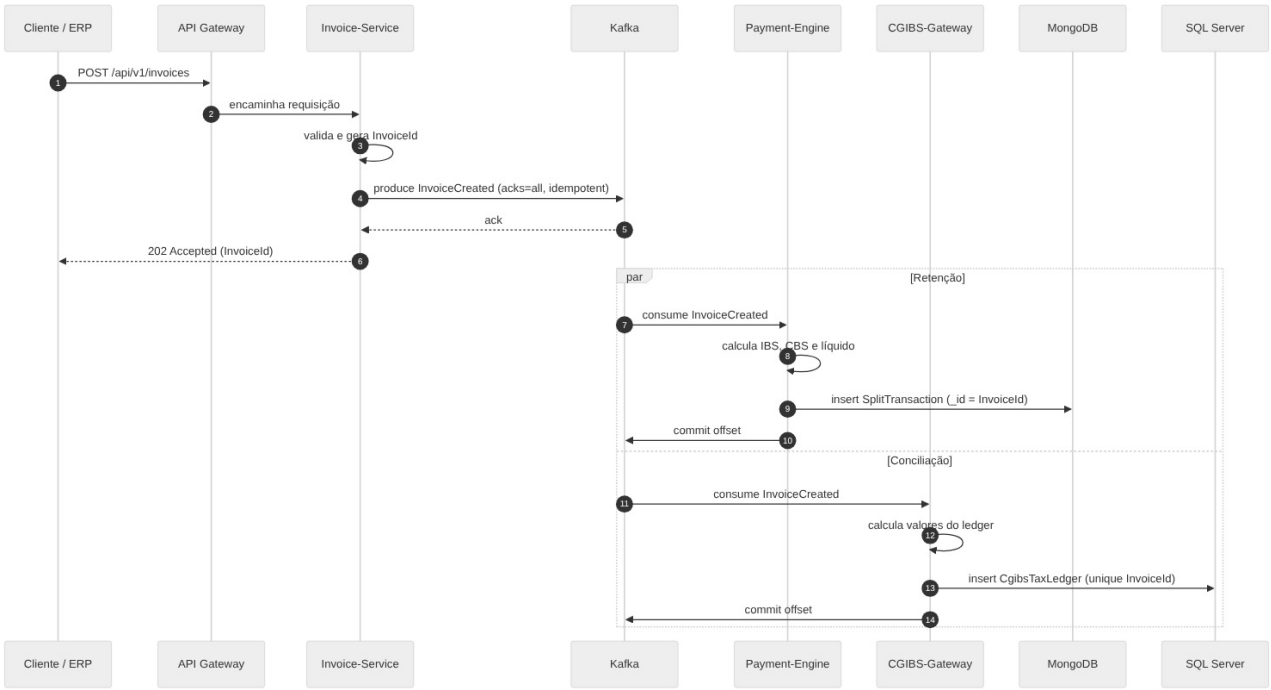
3. Componentes e responsabilidades

Serviço	Stack	Perfil de I/O	Responsabilidade
Invoice-Service	.NET 8 Web API	Stateless, RAM bound	Receber o pedido de emissão e publicar `InvoiceCreated`
Kafka	Apache Kafka	I/O sequencial em disco (PageCache)	Barramento de eventos e desacoplamento de concorrência
Payment-Engine	.NET 8 Worker	CPU bound, I/O assíncrono	Calcular IBS/CBS e o valor líquido (Art. 31/32)
CGIBS-Gateway	.NET 8 Worker	Database bound (relacional)	Auditoria e conciliação com o Comitê Gestor (Art. 471-D)
MongoDB	Mongo 7.0	B-Tree / WAL	Histórico flexível dos payloads de split
SQL Server	MSSQL 2022	ACID	Ledger fiscal do CGIBS

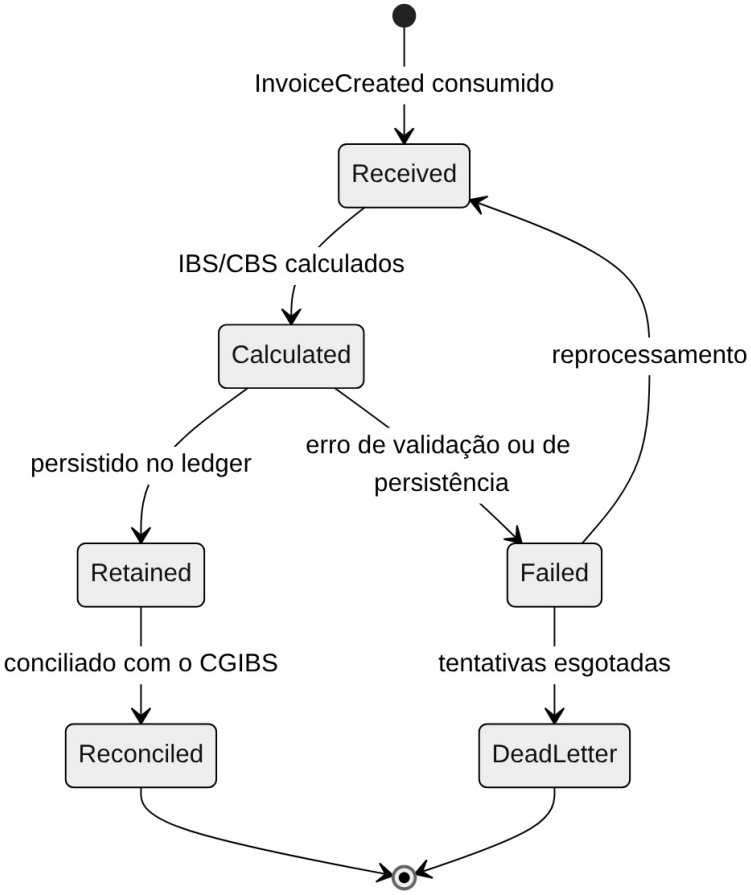
---

4. Fluxo de processamento

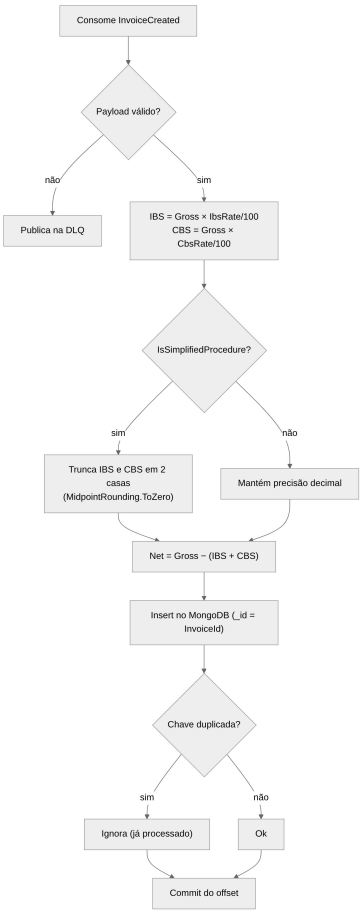
4.1 Sequência principal



4.2 Estados do ledger



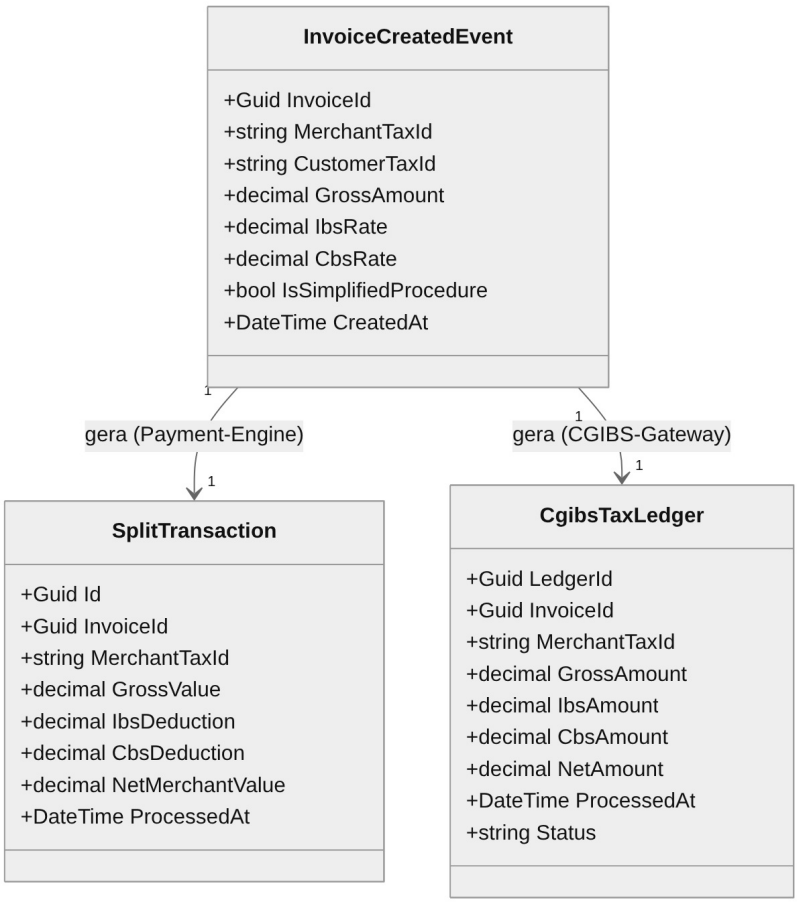
4.3 Algoritmo do Payment-Engine



---

5. Modelagem de dados

5.1 Modelo de domínio



5.2 Modelo relacional (CGIBS Ledger)

CgibsTaxLedger			
uniqueidentifier	LedgerId	PK	
uniqueidentifier	InvoiceId		UNIQUE
varchar14	MerchantTaxId		
decimal18_4	GrossAmount		
decimal18_4	IbsAmount		
decimal18_4	CbsAmount		
decimal18_4	NetAmount		
datetime2	ProcessedAt		
varchar20	Status		

```
CREATE TABLE CgibsTaxLedger (
    LedgerId       UNIQUEIDENTIFIER PRIMARY KEY,
    InvoiceId      UNIQUEIDENTIFIER NOT NULL,
    MerchantTaxId  VARCHAR(14)      NOT NULL,
    GrossAmount    DECIMAL(18, 4)   NOT NULL,
    IbsAmount      DECIMAL(18, 4)   NOT NULL,
    CbsAmount      DECIMAL(18, 4)   NOT NULL,
    NetAmount      DECIMAL(18, 4)   NOT NULL,
    ProcessedAt    DATETIME2        NOT NULL,
    Status         VARCHAR(20)      NOT NULL,
    CONSTRAINT UQ_CgibsTaxLedger_Invoice UNIQUE (InvoiceId),
    CONSTRAINT CK_CgibsTaxLedger_Sum CHECK (GrossAmount = IbsAmount + CbsAmount + NetAmount)
);
CREATE INDEX IX_CgibsTaxLedger_Merchant ON CgibsTaxLedger(MerchantTaxId);
```

> A constraint `UQ_..._Invoice` e o `CHECK` de soma são **adições** ao schema original: garantem idempotência e integridade aritmética no banco.

5.3 Documento MongoDB (`SplitPaymentDb.Transactions`)

```
{
  "_id": "3f1c0c9e-8a43-4b0e-9a5d-1d7f0a6b2c11",
  "merchantTaxId": "12345678000199",
  "grossValue": 1000.00,
  "ibsDeduction": 17.70,
  "cbsDeduction": 8.80,
  "netMerchantValue": 973.50,
  "processedAt": "2026-10-28T16:36:00Z"
}
```

Usar `InvoiceId` como `_id` torna o insert idempotente. Em C#, o `Guid` deve ser mapeado com representação padrão

(`GuidRepresentation.Standard`).

---

6. Contratos (API e eventos)

6.1 API REST

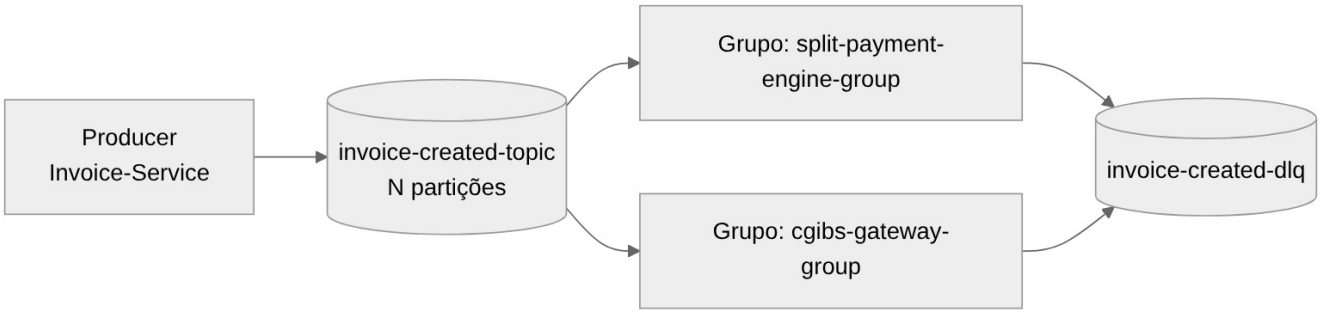
`POST /api/v1/invoices`

```
        {
          "merchantTaxId": "12345678000199",
          "customerTaxId": "98765432000155",
          "grossAmount": 1000.00,
          "ibsRate": 17.7,
          "cbsRate": 8.8,
          "isSimplifiedProcedure": false
        }
```

Código	Significado
`202 Accepted`	Evento publicado; retorna `{ invoiceId, status }`
`400 Bad Request`	Validação falhou (valor ≤ 0, alíquotas inválidas, CNPJ/CPF inválido)
`503 Service Unavailable`	Kafka indisponível

6.2 Evento `InvoiceCreated`

- **Tópico:** `invoice-created-topic`
- **Chave da mensagem:** `MerchantTaxId` (ordenação por fornecedor na partição)
- **Valor:** JSON do `InvoiceCreatedEvent`
- **Versionamento:** campo `schemaVersion` (sugerido) e evolução compatível



---

7. Regra de cálculo do Split

```
ibs   = gross × ibsRate / 100
cbs   = gross × cbsRate / 100
se IsSimplifiedProcedure: ibs, cbs = truncar(ibs, 2), truncar(cbs, 2)
net   = gross − (ibs + cbs)
```

Entrada	Valor
gross	1.000,00
ibsRate	17,7 %
cbsRate	8,8 %
IBS	177,00
CBS	88,00
Líquido	**735,00**

Os percentuais acima são ilustrativos. As alíquotas reais dependem do período de transição e de cada operação.

---

8. Infraestrutura (Docker Compose)

```yaml
version: '3.8'

services:
  zookeeper:
    image: confluentinc/cp-zookeeper:7.5.0
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
      ZOOKEEPER_TICK_TIME: 2000
    ports:
      - "2181:2181"

  kafka:
    image: confluentinc/cp-kafka:7.5.0
    depends_on:
      - zookeeper
    ports:
      - "9092:9092"
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:29092,PLAINTEXT_HOST://localhost:9092
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT
      KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1

  mongodb:
    image: mongo:7.0
    ports:
      - "27017:27017"
    environment:
      MONGO_INITDB_DATABASE: SplitPaymentDb

  sqlserver:
    image: mcr.microsoft.com/mssql/server:2022-latest
    ports:
      - "1433:1433"
    environment:
      SA_PASSWORD: "Your_password123!"
      ACCEPT_EULA: "Y"
```

Notas:

- Para o Kafka, o listener `PLAINTEXT_HOST` (`localhost:9092`) serve clientes no host; `PLAINTEXT` (`kafka:29092`) serve contêineres na mesma rede.
- `SA_PASSWORD` está obsoleta em favor de `MSSQL_SA_PASSWORD`; troque a senha e não a versione (use `.env`).
- O Kafka em modo KRaft (sem Zookeeper) é a opção moderna; a imagem 7.5.0 ainda suporta Zookeeper.

---

9. Resiliência, idempotência e consistência



Mecanismo	Onde	Efeito
`acks=all` + `enable.idempotence=true`	Producer	Sem perda e sem duplicidade por retentativa
Commit manual de offset	Consumers	Garantia *at-least-once*
PK / `_id` = `InvoiceId`	Mongo e SQL	Reprocessamento não duplica a retenção
Dead Letter Queue	Consumers	Mensagens venenosas não bloqueiam a partição
Chave de partição = `MerchantTaxId`	Producer	Ordem por fornecedor

Semântica efetiva: *at-least-once* na entrega + escrita idempotente = efeito *exactly-once* no armazenamento.

Consistência entre os dois destinos: Mongo e SQL são gravados por consumidores independentes, portanto são **eventualmente consistentes**. Um job de reconciliação periódico compara os totais por `InvoiceId`.

---

10. Implementação de referência (.NET 8)

10.1 Estrutura sugerida da solution

```
SplitPayment/
├── docker-compose.yml
├── README.md
├── src/
│   ├── SplitPayment.Domain/          # eventos, regras de cálculo
│   ├── SplitPayment.InvoiceApi/      # Web API (producer)
│   ├── SplitPayment.EngineWorker/    # consumer + MongoDB
│   └── SplitPayment.CgibsGateway/    # consumer + SQL Server
└── tests/
    └── SplitPayment.Tests/
```

10.2 Evento de domínio

```
namespace SplitPayment.Domain.Events;

public record InvoiceCreatedEvent
{
    public Guid InvoiceId { get; init; }
    public string MerchantTaxId { get; init; } = string.Empty;
    public string CustomerTaxId { get; init; } = string.Empty;
    public decimal GrossAmount { get; init; }
    public decimal IbsRate { get; init; }
    public decimal CbsRate { get; init; }
    public bool IsSimplifiedProcedure { get; init; }
    public DateTime CreatedAt { get; init; }
}
```

10.3 Worker de retenção (Payment-Engine)

```
using System.Text.Json;
using Confluent.Kafka;
using Microsoft.Extensions.Hosting;
using Microsoft.Extensions.Logging;
using MongoDB.Driver;
using SplitPayment.Domain.Events;

namespace SplitPayment.EngineWorker;

public class SplitPaymentConsumer : BackgroundService
{
    private readonly ILogger<SplitPaymentConsumer> _logger;
    private readonly IMongoCollection<SplitTransaction> _mongoCollection;
    private readonly ConsumerConfig _kafkaConfig;

    public SplitPaymentConsumer(ILogger<SplitPaymentConsumer> logger, IMongoClient mongoClient)
    {
        _logger = logger;
        _mongoCollection = mongoClient.GetDatabase("SplitPaymentDb")
            .GetCollection<SplitTransaction>("Transactions");

        _kafkaConfig = new ConsumerConfig
        {
            BootstrapServers = "localhost:9092",
            GroupId = "split-payment-engine-group",
            AutoOffsetReset = AutoOffsetReset.Earliest,
            EnableAutoCommit = false
        };
    }

    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
```

```
        {
            using var consumer = new ConsumerBuilder<Ignore, string>(_kafkaConfig).Build();
            consumer.Subscribe("invoice-created-topic");

            while (!stoppingToken.IsCancellationRequested)
            {
                try
                {
                    var consumeResult = consumer.Consume(stoppingToken);
                    if (consumeResult?.Message?.Value == null) continue;

                    var invoiceEvent = JsonSerializer.Deserialize<InvoiceCreatedEvent>(consumeResult.Message.Value);
                    if (invoiceEvent != null)
                    {
                        var transaction = ProcessSplit(invoiceEvent);
                        await _mongoCollection.InsertOneAsync(transaction, cancellationToken: stoppingToken);
                        consumer.Commit(consumeResult);
                        _logger.LogInformation("Split processado via LC 214/2025 Art. 31 para Invoice: {Id}",
invoiceEvent.InvoiceId);
                    }
                }
                catch (Exception ex)
                {
                    _logger.LogError(ex, "Erro no processamento da fila de Split Payment.");
                }
            }
        }

        private static SplitTransaction ProcessSplit(InvoiceCreatedEvent invoice)
        {
            decimal ibsValue = invoice.GrossAmount * (invoice.IbsRate / 100m);
            decimal cbsValue = invoice.GrossAmount * (invoice.CbsRate / 100m);

            if (invoice.IsSimplifiedProcedure)
            {
                ibsValue = Math.Round(ibsValue, 2, MidpointRounding.ToZero);
                cbsValue = Math.Round(cbsValue, 2, MidpointRounding.ToZero);
            }

            decimal totalTax = ibsValue + cbsValue;
            decimal netMerchantAmount = invoice.GrossAmount - totalTax;

            return new SplitTransaction
            {
                Id = Guid.NewGuid(),
                InvoiceId = invoice.InvoiceId,
                MerchantTaxId = invoice.MerchantTaxId,
                GrossValue = invoice.GrossAmount,
                IbsDeduction = ibsValue,
                CbsDeduction = cbsValue,
                NetMerchantValue = netMerchantAmount,
                ProcessedAt = DateTime.UtcNow
            };
        }
    }

    public class SplitTransaction
    {
        public Guid Id { get; set; }
        public Guid InvoiceId { get; set; }
        public string MerchantTaxId { get; set; } = string.Empty;
        public decimal GrossValue { get; set; }
        public decimal IbsDeduction { get; set; }
        public decimal CbsDeduction { get; set; }
        public decimal NetMerchantValue { get; set; }
        public DateTime ProcessedAt { get; set; }
    }
```

> Esta é a versão do blueprint original. As correções recomendadas (idempotência real, DLQ, producer singleton) estão na seção 12.

10.4 API emissora (Invoice-Service)

```
using System.Text.Json;
using Confluent.Kafka;
using Microsoft.AspNetCore.Builder;
using Microsoft.AspNetCore.Http;

var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.MapPost("/api/v1/invoices", async (InvoiceRequest request) =>
{
    var config = new ProducerConfig { BootstrapServers = "localhost:9092" };
    using var producer = new ProducerBuilder<Null, string>(config).Build();

    var eventPayload = new
    {
        InvoiceId = Guid.NewGuid(),
        request.MerchantTaxId,
        request.CustomerTaxId,
        request.GrossAmount,
        request.IbsRate,
        request.CbsRate,
        request.IsSimplifiedProcedure,
        CreatedAt = DateTime.UtcNow
    };

    var message = JsonSerializer.Serialize(eventPayload);
    var result = await producer.ProduceAsync("invoice-created-topic",
        new Message<Null, string> { Value = message });

    return Results.Accepted($"/api/v1/invoices/{eventPayload.InvoiceId}",
        new { eventPayload.InvoiceId, Status = "Enviado para Split" });
});

app.Run();

public record InvoiceRequest(
    string MerchantTaxId,
    string CustomerTaxId,
    decimal GrossAmount,
    decimal IbsRate,
    decimal CbsRate,
    bool IsSimplifiedProcedure
);
```

---

11. Performance e baixo nível

Operação	Tempo	Espaço	Observação de alocação
Kafka produce	O(1) amortizado	O(N) buffer	Gen 0, baixo impacto
Cálculo do split	O(1)	O(1)	Poucas alocações (`decimal` é struct)
Insert MongoDB	O(log N)	O(N)	Índice B-Tree; LOH se JSON grande
Insert SQL (índice)	O(log N)	O(N)	Páginas no Buffer Pool

Aritmética monetária. `decimal` em C# usa 128 bits com fator de escala em base 10, o que evita os erros de representação do IEEE 754 (`float`/`double`). Esse é o motivo da escolha para valores fiscais.

Kafka e disco. O broker grava sequencialmente no log e se beneficia do PageCache do sistema operacional. O `sendfile()` (zero-copy) é usado pelo **broker ao servir leituras aos consumers**, não na escrita do producer.

---

12. Pontos de atenção e melhorias

Revisão técnica do blueprint original, com correções sugeridas.

#	Ponto	Problema	Correção recomendada
1	Idempotência	O worker usa `Id = Guid.NewGuid()` como chave; a duplicidade **não** é bloqueada, ao contrário do que o texto afirma	Usar `Id = InvoiceId` (ou índice único em `InvoiceId`) e tratar `DuplicateKey` como sucesso
2	Producer por requisição	Criar e descartar um `IProducer` a cada chamada é caro	Registrar um `IProducer` singleton na DI
3	Config do producer	Faltam `Acks.All` e `EnableIdempotence = true` no código, citados só na documentação	Incluí-los no `ProducerConfig`
4	Chave da mensagem	`Null` como chave distribui sem ordenação	Usar `MerchantTaxId` como chave
5	Tratamento de erro	O `catch` genérico faz o loop reprocessar a mesma mensagem indefinidamente	Retry limitado + DLQ
6	CGIBS Gateway	Está na topologia, mas **sem implementação**	Criar o worker com SQL Server (item 10 do roadmap)
7	Arredondamento	Trunca só no modo simplificado; no modo normal o valor pode ter mais de 2 casas	Definir uma regra única de arredondamento monetário
8	Reconciliação	`Net = Gross − tax` pode divergir do soma de partes se houver arredondamento	`CHECK` de soma no ledger (seção 5.2)
9	`sendfile()`	O texto original o atribui ao producer	Corrigido na seção 11
10	Segredos	Senha do SQL Server fixa no compose	`.env` e secrets
11	Validação	A API aceita qualquer valor	Validar `GrossAmount > 0`, alíquotas 0–100 e documentos fiscais
12	Referências legais	Artigos (inclusive 471-D) não verificados	Revisão jurídica

Exemplo de worker idempotente

```
        try
        {
            await _mongoCollection.InsertOneAsync(transaction, cancellationToken: stoppingToken);
        }
        catch (MongoWriteException ex) when (ex.WriteError.Category == ServerErrorCategory.DuplicateKey)
        {
            _logger.LogWarning("Invoice {Id} já processada; ignorando.", invoiceEvent.InvoiceId);
        }
        consumer.Commit(consumeResult);
```

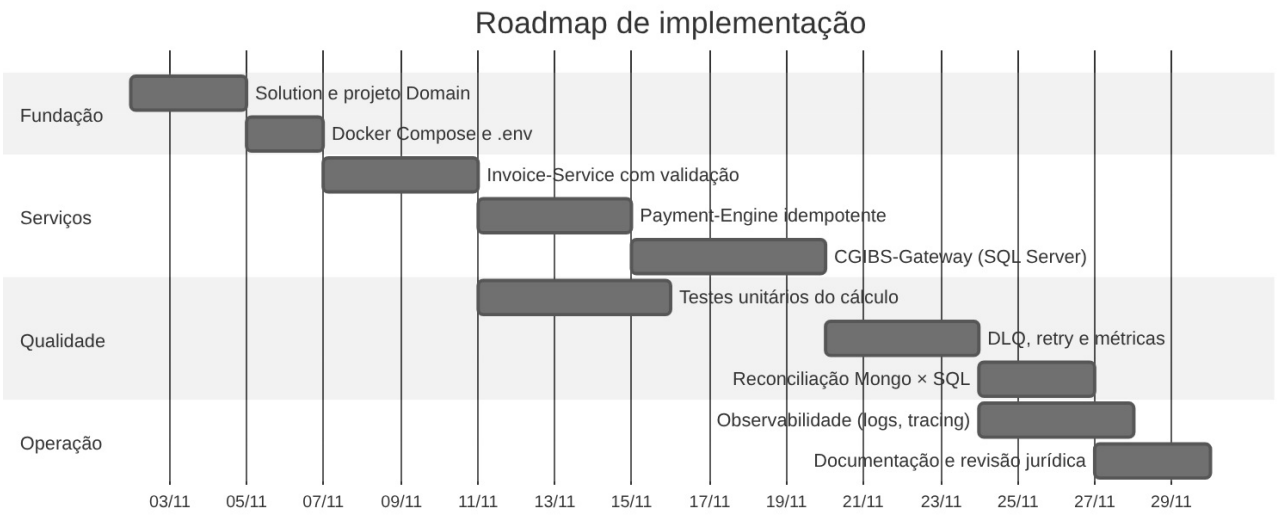
com `Id = invoice.InvoiceId` em `ProcessSplit`.

Exemplo de producer singleton

```
        builder.Services.AddSingleton<IProducer<string, string>>(_ =>
            new ProducerBuilder<string, string>(new ProducerConfig
            {
                BootstrapServers = builder.Configuration["Kafka:Bootstrap"],
                Acks = Acks.All,
                EnableIdempotence = true
            }).Build());
```

---

13. Roadmap



Como executar

```
        docker compose up -d
        dotnet run --project src/SplitPayment.InvoiceApi
        dotnet run --project src/SplitPayment.EngineWorker
        dotnet run --project src/SplitPayment.CgibsGateway

        curl -X POST http://localhost:5000/api/v1/invoices \
          -H "Content-Type: application/json" \
          -d
'{"merchantTaxId":"12345678000199","customerTaxId":"98765432000155","grossAmount":1000.00,"ibsRate":17.7,"cbsRate":8.8
,"isSimplifiedProcedure":false}'
```